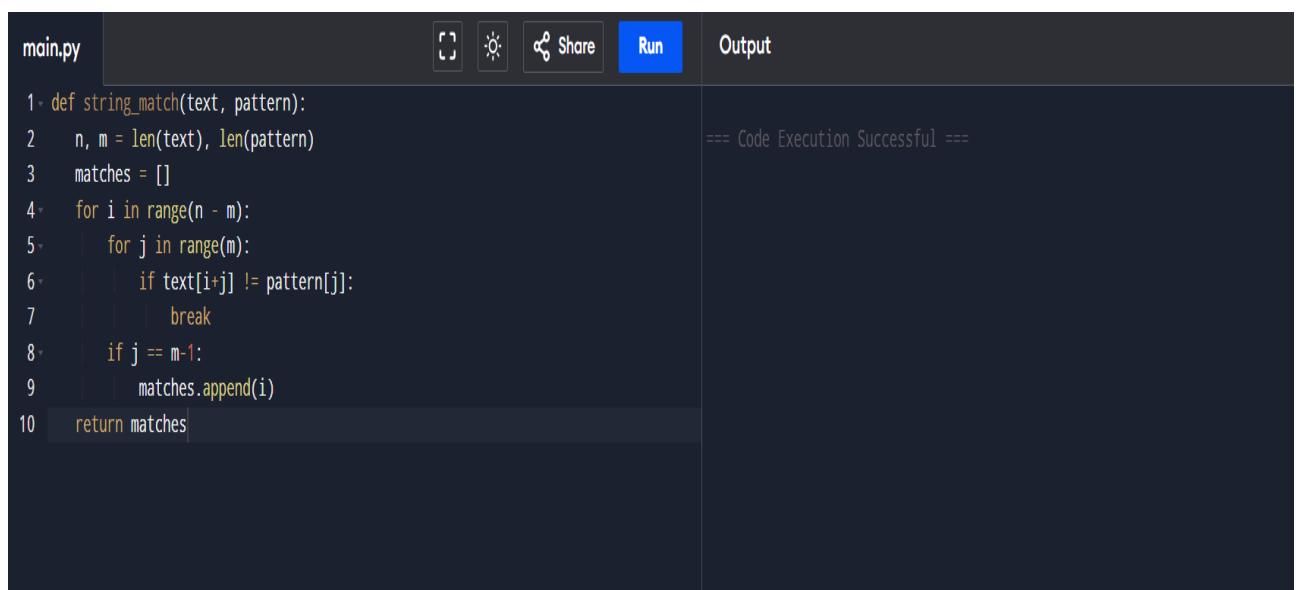


Q27. Debugging String Matching Code (Skipped Last Valid Position)

The following brute force string matching implementation fails to examine the last valid starting position of the pattern in the text. Carefully inspect the loop bounds and explain why one potential match is always skipped. Debug the code so that every valid alignment is checked correctly. Optimize the implementation for clarity while preserving brute force behavior. Analyze the time complexity of the corrected algorithm. Rewrite the final improved code with proper comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            matches.append(i)
    return matches
```

Original code and execution:



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 def string_match(text, pattern):
2     n, m = len(text), len(pattern)
3     matches = []
4     for i in range(n - m):
5         for j in range(m):
6             if text[i+j] != pattern[j]:
7                 break
8         if j == m-1:
9             matches.append(i)
10    return matches
```

The 'Run' button is highlighted in blue. The 'Output' panel on the right shows the message: '=== Code Execution Successful ==='.

Step 1: Understand the Problem

The brute-force string matching algorithm should compare the pattern with every possible position in the text.

Number of possible alignments:

$$\text{Number of Alignments} = n - m + 1$$

where

- n = length of text

- m = length of pattern

Step 2: Identify the Error

Error Found

for i in range($n - m$):

Why is this an Error?

The outer loop determines the starting position of the pattern in the text. If the text length is n and the pattern length is m , the last valid starting position is:

$$n - m$$

However,
range($n - m$)

generates

0, 1, 2, ..., $n - m - 1$

The value $n - m$ is never included because Python's range() excludes the ending value.

As a result, the algorithm skips the last valid alignment, causing valid matches at the end of the text to be missed.

Example

Text = "ABCDE"

Pattern = "DE"

Length of Text (n) = 5

Length of Pattern (m) = 2

Last Valid Starting Position = $5 - 2 = 3$

Possible alignments

Starting Position	Substring	Checked
0	AB	✓ Yes
1	BC	✓ Yes
2	CD	✓ Yes
3	DE	✗ No

The match at index **3** is skipped because the loop never reaches that position.

Step 3: Correct the Loop Boundary

Modification

Replace

```
for i in range(n - m):
```

with

```
for i in range(n - m + 1):
```

Updated Code

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
    matches = []  
    # Fixed outer loop  
    for i in range(n - m + 1):  
        for j in range(m):  
            if text[i+j] != pattern[j]:  
                break  
        if j == m-1:  
            matches.append(i)  
    return matches
```

Explanation

Adding **+1** ensures that the last valid starting position is included in the search. Now the algorithm checks every possible alignment between the text and the pattern, eliminating the logical error.

```
main.py  [Icons]  Share  Run  Output
1 def string_match(text, pattern):
2     n, m = len(text), len(pattern)
3     matches = []
4     for i in range(n - m + 1):
5         for j in range(m):
6             if text[i+j] != pattern[j]:
7                 break
8             if j == m-1:
9                 matches.append(i)
10    return matches

=== Code Execution Successful ===
```

Step 4: Improve the Match Detection Logic

Current Logic

```
if j == m-1:
    matches.append(i)
```

Why Improve It?

Although this condition works correctly, it depends on the value of `j` after the loop terminates. This makes the program slightly harder to understand, especially for beginners. Using a Boolean variable improves readability and makes the intention of the code much clearer.

Improved Code

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m + 1):
        found = True
        for j in range(m):
            if text[i+j] != pattern[j]:
                found = False
                break
        if found:
            matches.append(i)
```

```
return matches
```

Explanation

The Boolean variable `found` assumes that the pattern matches initially. If any mismatch occurs, it becomes `False`. After the inner loop completes, the pattern is added to the result only if `found` remains `True`.

Step 5: Final Code Optimization

Reason for Optimization

Python provides a cleaner feature called `for-else`, where the `else` block executes only if the loop finishes without encountering a `break`. This removes the need for an extra Boolean variable and makes the code shorter and more readable.

Final Improved Code

```
def string_match(text, pattern):  
    # Find lengths of text and pattern  
    n = len(text)  
    m = len(pattern)  
    # Store matching positions  
    matches = []  
    # Check every possible alignment  
    for i in range(n - m + 1):  
        # Compare each character  
        for j in range(m):  
            # Stop if mismatch occurs  
            if text[i + j] != pattern[j]:  
                break  
        # Executes only when the entire pattern matches  
        else:  
            matches.append(i)  
    return matches  
  
# Example  
text = "ABABCABC"  
pattern = "ABC"  
print(string_match(text, pattern))
```

Output:

main.py	Output
<pre>1 def string_match(text, pattern): 2 n = len(text) 3 m = len(pattern) 4 matches = [] 5 for i in range(n - m + 1): 6 for j in range(m): 7 if text[i + j] != pattern[j]: 8 break 9 else: 10 matches.append(i) 11 return matches 12 text = "ABABCABC" 13 pattern = "ABC" 14 print(string_match(text, pattern))</pre>	<pre>[2, 5] === Code Execution Successful ===</pre>

Time Complexity Analysis

Best Case

The first character mismatches for every alignment.

Time Complexity: $O(n)$

Worst Case

Every alignment requires comparing all m characters.

Time Complexity: $O(n \times m)$

Average Case

Most alignments require multiple comparisons.

Time Complexity: $O(n \times m)$

Space Complexity

Only a few variables are used apart from the output list.

Auxiliary Space: $O(1)$

Brute Force String Matching Algorithm

Introduction

The Brute Force String Matching Algorithm, also known as the Naive String Matching Algorithm, is one of the simplest methods used to find a pattern within a text. The algorithm checks whether the pattern matches the text at every possible starting position. It compares the characters of the pattern with the corresponding characters in the text one by one. If all

characters match, the starting index is recorded; otherwise, the algorithm shifts the pattern by one position and repeats the comparison. Although this algorithm is easy to understand and implement, it is not the most efficient for large texts because it may compare the same characters multiple times. However, it serves as the foundation for understanding more advanced string matching algorithms such as Knuth-Morris-Pratt (KMP), Boyer-Moore, and Rabin-Karp.

Working Principle

The brute force string matching algorithm follows these steps:

1. Place the pattern at the beginning of the text.
2. Compare each character of the pattern with the corresponding character in the text.
3. If all characters match, record the starting index of the match.
4. If a mismatch occurs, shift the pattern one position to the right.
5. Repeat the process until all possible alignments have been checked.

Algorithm

1. Read the input text and pattern.
2. Calculate the lengths of the text (n) and pattern (m).
3. For each possible starting position from 0 to $n - m$, compare the pattern with the corresponding substring of the text.
4. If every character matches, store the starting position.
5. Continue until all alignments are checked.
6. Return the list of matching positions.

Applications

The Brute Force String Matching Algorithm is widely used in:

- Text editors for searching words or phrases.
- Search engines for basic pattern searching.
- DNA and genome sequence matching.
- Plagiarism detection systems.
- Log file and document analysis.
- Basic pattern searching in programming applications.

Advantages

- Very simple to understand and implement.
- Does not require any preprocessing of the pattern.
- Suitable for small text and pattern sizes.
- Easy to debug and modify.

Disadvantages

- Inefficient for large texts.
- Performs many unnecessary character comparisons.

- Worst-case time complexity is **$O(n \times m)$** .
- Slower than advanced algorithms like KMP and Boyer-Moore.